

Plano de Desenvolvimento — VAIJUNTO

Sistema de caronas compartilhadas (PBL de Redes) — plano pensado para quem está começando do zero em Docker, servidores, TCP/IP e protocolos de rede.

Ponto de partida

- Nunca mexeu com servidor, socket TCP/IP ou protocolo de rede.
- Nunca usou Docker.
- Boa base em lógica de programação e algoritmos.
- Vai aprender Go pela primeira vez neste projeto.
- Prazo oficial de entrega: 17/09/2026, apresentação final em 22/09/2026.
- Meta pessoal: **sistema completo e funcional até 10/09/2026**, deixando a semana de 10/09 a 17/09 livre para documentação, relatório e ensaios, sem código novo sob pressão. Isso exige estudar também fora das sessões de desenvolvimento da disciplina.

Por isso, o plano avança em degraus pequenos: cada fase introduz **um conceito novo por vez** e termina em algo que roda e que dá pra testar, antes de passar pra próxima. Nunca duas coisas desconhecidas ao mesmo tempo (ex: não aprendemos concorrência e Docker juntos).

Decisões já tomadas (e por quê)

Decisão	Escolha	Motivo
Linguagem	Go	Concorrência é o ponto forte da linguagem (goroutines/channels), binário único sem dependências, roda igual em Windows/Linux, pacote net já dá socket TCP puro sem framework externo.
Formato das mensagens	JSON + prefixo de tamanho (framing)	Fácil de debugar e inspecionar, biblioteca padrão do Go resolve, interoperava com qualquer linguagem — como o enunciado exige.
Controle de concorrência	Mutex por carona	Cada carona tem seu próprio "cadeado"; simples de implementar e de explicar no relatório.
Interface dos clientes	Terminal (CLI)	Evita a complicação de rodar interface gráfica dentro de um container Docker (exigiria X11 forwarding). Todo o esforço fica na rede/concorrência, que é o que está sendo avaliado.
Persistência	Em memória (sem banco de dados)	O enunciado proíbe delegar controle de concorrência a um banco; o servidor mantém tudo em estruturas na RAM enquanto roda.

Glossário rápido (consulte sempre que precisar)

- **Socket:** o "encaixe" pelo qual dois programas em máquinas diferentes trocam bytes pela rede. É a peça básica sobre a qual tudo será construído.
- **TCP:** protocolo que garante que os bytes cheguem completos, na ordem certa, sem duplicar. Pense nele como um "cano" confiável entre cliente e servidor — mas sem noção de onde uma mensagem começa/termina (isso é o *framing*, que definimos nós).
- **Framing:** como delimitamos onde uma mensagem acaba dentro do fluxo contínuo de bytes do TCP. Nossa escolha: antes de cada mensagem JSON, mandamos 4 bytes dizendo o tamanho dela.
- **Goroutine:** uma "tarefa leve" do Go que roda concorrentemente com outras. Usaremos uma goroutine por cliente conectado, para o servidor atender vários ao mesmo tempo.
- **Race condition:** bug que ocorre quando duas execuções concorrentes mexem no mesmo dado ao mesmo tempo e o resultado final fica errado/inconsistente (ex: dois passageiros reservando o último assento e os dois "conseguindo").
- **Mutex:** um cadeado que garante que só uma goroutine mexe em um dado por vez, evitando race conditions.
- **Container (Docker):** uma "caixa" que empacota o programa já compilado, pronta pra rodar em qualquer máquina sem precisar instalar nada além do Docker.

Calendário (entregável de código concreto por data)

Data	Sessão	Entregável concreto
24/08 (hoje)	—	Início por conta própria — não esperar a sessão de tutoria pra começar a Fase 0
25/08	3 (Tutorial)	Fase 0 concluída: consegue escrever/ler <code>struct</code> , <code>slice</code> , <code>map</code> , função com <code>error</code> , sem consultar a cada linha (exercícios soltos, sem rede ainda)
27/08	4 (Dev)	Fase 1 concluída: servidor TCP "eco" rodando + cliente que conecta e troca mensagens de texto — testado com 2 clientes ao mesmo tempo
01/09	5 (Tutorial)	Fase 2 concluída: pacote protocolo/ pronto (<code>structs</code> de login, publicar carona, buscar itinerário, reservar, cancelar + <code>EnviarMensagem/ReceberMensagem</code> com framing), testado entre dois processos. Início do modelo de domínio (<code>struct Carona</code> com assentos por trecho)
03/09	6 (Dev)	Fase 3 concluída: servidor sequencial completo — publicar / consultar / reservar corretos contra 1 cliente de teste. Goroutine por conexão implementada (aceita várias conexões simultâneas)

08/09	7 (Tutorial)	Fase 4 concluída: mutex por carona + reserva atômica de itinerário multi-trecho, testado com 2+ clientes concorrentes disputando o mesmo trecho sem duplicar assento. Menu do cliente motorista terminado (login, publicar, listar caronas/passageiros confirmados, cancelar) funcionando ponta a ponta
10/09	8 (Dev)	Menu do cliente passageiro terminado (login, buscar itinerário, reservar, consultar/cancelar) + Dockerfile do servidor e dos clientes buildando e rodando + programa de teste automatizado com N clientes simultâneos rodando e reportando resultado. Sistema completo e funcional
10/09 → 17/09	—	Sem código novo: documentação do protocolo (Fase 8), relatório SBC (Fase 9), teste em duas máquinas do laboratório, ensaios — folga para imprevistos
17/09	Entrega	Submissão final
22/09	Apresentação	Apresentações finais

Use as sessões de Tutorial (25/08, 01/09, 08/09) para tirar dúvidas com o professor/tutor exatamente sobre o conceito novo daquele trecho — são os pontos mais conceituais (sintaxe de Go, desenho do protocolo, concorrência).

Fase 0 — Fundamentos de Go (até 25/08)

Objetivo: ganhar fluência mínima na linguagem antes de tocar em rede.

O que aprender: variáveis e tipos, `struct`, `slice/map`, funções e retorno de erro (`if err != nil`), `interface` básica, e uma introdução a goroutines (`go func(){...}()`) sem se aprofundar ainda.

O que construir: 2-3 exercícios pequenos e isolados do projeto (ex: uma `struct Carona` com campos e um `slice` de `Caronas`, uma função que filtra caronas por cidade). Nada de rede ainda.

Pronto quando: você lê um programa Go simples e entende o que cada linha faz, sem precisar consultar a cada instrução.

Fase 1 — Sockets TCP básicos (até 27/08)

Objetivo: entender o que é um servidor de verdade, na prática mais simples possível.

O que aprender: `net.Listen`, `Accept`, `Read/Write` em uma conexão, o *accept loop* (`for { conn := listener.Accept(); go handle(conn) }`), por que cada conexão vira uma goroutine.

O que construir: um servidor "eco" (devolve exatamente o que o cliente manda) e um cliente que conecta, manda uma linha de texto e imprime a resposta. Testar com dois clientes ao mesmo tempo.

Pronto quando: você consegue explicar, com suas palavras, o caminho que um byte percorre do cliente até o servidor e de volta.

Fase 2 — Protocolo de aplicação (até 01/09)

Objetivo: desenhar a "linguagem" que motorista, passageiro e servidor vão falar entre si.

O que definir: - As operações: publicar carona, consultar caronas/itinerários, reservar itinerário, cancelar carona, cancelar reserva. - O formato de cada mensagem (structs Go, ex: `type PublicarCaronaRequest struct { Rota []string; Data string; Assentos int; PrecoPorTrecho map[string]float64 }`), serializadas em JSON. - O framing: função `EnviarMensagem(conn, msg)` que escreve [4 bytes tamanho][JSON], e `ReceberMensagem(conn)` que lê o tamanho e depois os bytes exatos.

O que construir: um pacote Go compartilhado (protocolo/) com essas structs e as funções de enviar/receber, usado tanto pelo servidor quanto pelos clientes.

Pronto quando: você tem um documento (ainda que rascunho) listando cada tipo de mensagem e seus campos — isso vira depois a documentação exigida no enunciado.

Fase 3 — Modelo de domínio + servidor sequencial (até 03/09)

Objetivo: a lógica de negócio funcionando, ainda sem se preocupar com concorrência (um cliente por vez).

O que aprender: como representar disponibilidade de assento *por trecho* (não por carona inteira) — ex: uma matriz ou mapa de trechos consecutivos com contagem de assentos livres.

O que construir: servidor que aceita uma conexão, processa os pedidos dessa conexão em sequência (publicar carona, consultar, reservar), atualiza o estado em memória, responde. Ainda sem goroutines concorrentes disputando o mesmo dado.

Pronto quando: um cliente de teste consegue publicar uma carona com trechos e reservar um itinerário simples, e o servidor recusa reservas que excedem os assentos livres do trecho.

Fase 4 — Concorrência (até 08/09)

Objetivo: o núcleo do que a disciplina quer te ensinar: lidar com múltiplos clientes disputando o mesmo recurso.

O que aprender: goroutine por conexão de verdade (várias conexões simultâneas mexendo no mesmo estado), `sync.Mutex`, como reservar um itinerário com **vários trechos de motoristas diferentes de forma atômica** (se qualquer trecho falhar, desfazer os que já foram reservados nessa tentativa).

O que construir: cada `Carona` ganha seu próprio mutex; a reserva de um itinerário trava todas as caronas envolvidas (em uma ordem consistente, para evitar deadlock), confirma tudo ou desfaz tudo.

Pronto quando: você consegue rodar 2+ clientes ao mesmo tempo tentando reservar o último assento do mesmo trecho e só um consegue — sem o servidor travar ou corromper o estado.

Fase 5 — Clientes CLI completos (motorista até 08/09, passageiro até 10/09)

Objetivo: interfaces de terminal usáveis para motorista e passageiro.

O que construir: - Cliente motorista (até 08/09, junto da Fase 4): autenticar, publicar carona, listar caronas publicadas e passageiros confirmados por trecho, cancelar carona. - Cliente passageiro (até 10/09): autenticar, buscar itinerários entre origem/destino/data, confirmar reserva, consultar/cancelar reservas.

Ambos usando o pacote `protocolo/` da Fase 2 — nenhuma lógica de rede nova aqui, só a interface de menus no terminal.

Pronto quando: dá pra rodar uma demonstração manual completa (motorista publica → passageiro reserva → motorista vê o passageiro confirmado).

Fase 6 — Docker (até 10/09)

Objetivo: empacotar servidor e clientes para rodar em máquinas separadas no laboratório.

O que aprender: o que é uma imagem Docker, `Dockerfile` (receita de como construir a imagem), `docker build`, `docker run`, como mapear a porta do servidor (`-p`) para que outra máquina consiga se conectar.

O que construir: um `Dockerfile` simples para o servidor e um para os clientes (aproveitando o binário único do Go — a imagem fica pequena). Testar rodando servidor e cliente em containers na mesma máquina primeiro, depois (se possível) em duas máquinas do laboratório.

Pronto quando: `docker run` do servidor em uma máquina + `docker run -it` do cliente em outra conseguem se comunicar pela rede do laboratório.

Fase 7 — Teste automatizado de concorrência (até 10/09)

Objetivo: cumprir o requisito de um teste que sobe vários clientes simultâneos disputando os mesmos trechos.

O que construir: um programa Go (ou teste automatizado) que dispare N goroutines, cada uma agindo como um cliente completo, todas tentando reservar os mesmos trechos ao mesmo tempo; ao final, verifica que nenhum assento foi vendido duas vezes e nenhum itinerário ficou "pela metade", e mede o tempo de resposta.

Pronto quando: o teste roda contra o servidor real (via socket, não chamando funções internas) e reporta os resultados.

Fase 8 — Documentação do protocolo (até 17/09)

Consolidar o que foi rascunhado na Fase 2 num documento no repositório: cada operação, formato de mensagem, fluxo de conexão/desconexão, exemplos reais de mensagens trocadas (capturadas rodando o sistema).

Fase 9 — Relatório (formato SBC, máx. 8 páginas)

Escrever ao longo do processo, não só no fim — depois de cada fase, anote em 2-3 frases a decisão técnica tomada e por quê (isso já vira o corpo do relatório). Seções sugeridas: introdução ao problema, fundamentação teórica (sockets, concorrência), arquitetura da solução, protocolo, estratégia de concorrência, testes, conclusão.

Estrutura de pastas prevista

```
/servidor      -> main.go do servidor central
/cliente-motorista -> main.go do cliente motorista (CLI)
/cliente-passageiro -> main.go do cliente passageiro (CLI)
/protocolo      -> pacote Go compartilhado: structs de mensagens + framing
/teste-concorrencia -> programa de teste de carga
/Documentação
/Plano          -> este plano
protocolo.md    -> especificação do protocolo (Fase 8)
relatorio-sbc.pdf -> relatório final (Fase 9)
```

Checklist final (mapeado aos requisitos do enunciado)

- ☐ Servidor central com estado de caronas e reservas
- ☐ Cliente motorista (autenticar, publicar, consultar, cancelar)
- ☐ Cliente passageiro (autenticar, buscar itinerários, reservar, consultar/cancelar)
- ☐ Protocolo sobre socket TCP puro, sem framework de RPC/mensageria
- ☐ Reserva de itinerário atômica (tudo ou nada)
- ☐ Controle de concorrência implementado por nós (mutex), sem delegar a banco/serviço externo
- ☐ Servidor resiliente à queda abrupta de um cliente
- ☐ Especificação do protocolo documentada no repositório
- ☐ Teste automatizado com múltiplos clientes simultâneos
- ☐ Servidor e clientes containerizados, rodando em máquinas distintas no laboratório
- ☐ Relatório em formato SBC (máx. 8 páginas)
- ☐ Código no GitHub com README